

RELATÓRIO 05 — SANDBOX DE TESTES EVOLUTIVA

M.I.N.D. — Muñdji Intelligent Neural Developer
Instruções 45 a 61

Campo	Valor
Data	3 de Agosto de 2026
Commit	06a57a6
Suite	304 passam · 4 saltados (234 anteriores intactos + 70 novos)
Estado por omissão	SANDBOX_TESTS_ENABLED=false — desligada
Mudança central	A percentagem deixa de ser estimada pelo modelo e passa a ser medida sobre testes reais

1. O que fiz

Implementei a extensão na totalidade, conforme o *Prompt_Final_Sandbox* — que é a versão completa e supersede o *Sandbox_Evolutiva*. Comparei os dois antes de começar: a base conceptual é a mesma, mas o final traz as assinaturas de método, os métodos de base de dados, a lista de testes obrigatória, e uma diferença que não é de pormenor — **o default de SANDBOX_TESTS_ENABLED mudou de true para false**. Implementei o final.

1.1 As duas propriedades

Propriedade	Como ficou
Opção A — biblioteca que cresce entre ciclos	Os testes de um ciclo aprovado ficam na <code>test_library</code> com o embedding da tarefa. Um ciclo novo com tarefa semelhante herda-os por similaridade cosseno. Testes de ciclos reprovados nunca são herdados — seria propagar critérios que ainda não se provaram.
Opção B — rigor crescente dentro do ciclo	Iteração 1 → nível 1 (básico), iteração 2 → nível 2 (limite), iteração 3+ → nível 3 (erro). Acumulativos por omissão: na iteração 3 corre 1+2+3. Não basta não regredir, tem de progredir.

A separação de responsabilidades foi reforçada: o CORTEX não gera os testes pelos quais vai ser avaliado. Gera-os o CEREBELLUM, na transição Fase 1 → Fase 2, para que sejam informados pela intenção da tarefa e não moldados ao código que veio a ser gerado.

1.2 A percentagem passa a ser medida

É a mudança com mais consequência. Antes, a percentagem era o que o modelo achava depois de ver a sandbox rebentar ou não. Agora:

Situação	Percentagem
Nível 1 completo	33%
Níveis 1 e 2 completos	66%
Três níveis completos	99%
Qualidade do relatório (conformidade JSON)	+1% → 100%
Parcial	(passados / total) × 33 por nível

Situação	Percentagem
Nível sem testes	Contribui 0

A última linha é a que muda o comportamento do sistema: um código que passa todos os testes básicos mas nunca foi confrontado com testes de limite fica nos 33% e **não pode ser aprovado**. O número mostra a lacuna em vez de a esconder. O único juízo que resta ao modelo vale 1% — e é a conformidade com o esquema JSON, que é verificável.

2. Como fiz

Ficheiro	Papel
<i>test_generator.py</i> (novo)	Classe TestGenerator. Gera testes nos 3 níveis, consulta o HIPPOCAMPUS para herdar de ciclos aprovados, guarda na test_library. Os herdados entram no prompt como contexto para o modelo não duplicar.
<i>test_runner.py</i> (novo)	Classe TestRunner. Encapsula o sandbox.py sem o modificar — é o mesmo runner multi-linguagem, só recebe código de teste em vez de código de aplicação.
<i>database.py</i>	Tabelas test_library e test_results, mais save_test, record_test_result, get_tests_for_cycle, get_tests_by_embedding e mark_tests_permanent.
<i>ml_features.py</i>	embed_task (384 dimensões) e cosine_similarity, com fallback determinístico quando não há sentence-transformers.
<i>hippocampus.py</i>	recommend_tests, que funciona independentemente de ML_ENABLED — é isso que permite a biblioteca crescer desde o primeiro ciclo.
<i>cerebellum.py</i> , <i>cortex.py</i> , <i>graph.py</i>	Integração no fluxo, schema estendido, e o limite por nível.

2.1 Uma ambiguidade da especificação, e como a resolvi

O texto da Parte 2 diz que o CEREBELLUM gera os testes «*depois do código base estar aprimorado e com os marcadores anotados*». Mas o fluxo da Parte 4.2 diz que o **CORTEX recebe os testes como restrições ao refinar o código base** — e o refinamento acontece antes da anotação de marcadores. As duas coisas não podem ser ambas verdade.

Segui o fluxo da Parte 4.2, porque é o que torna a restrição possível: gerar depois da anotação tornaria impossível o CORTEX usá-los para refinar, que é o efeito mais valioso dos dois. A consequência é que os marcadores podem ainda não existir no momento da geração; nesse caso os testes saem com *neuron_target="all"* e linguagem python por omissão. Digo-o aqui em vez de escolher em silêncio.

2.2 Decisões de desenho que tive de tomar

- **Testes de falha esperada.** Um teste com *expected_outcome="fail"* conta como passado quando o código rebenta, e como falhado quando o código passa. Sem esta inversão, os testes de nível 3 — que existem para verificar que erros são levantados — nunca poderiam passar.
- **'error' e 'timeout' contam como falha** nas estatísticas da biblioteca. Não passaram, e é isso que interessa à taxa de sucesso.
- **O 1% da qualidade do relatório** é atribuído por o modelo ter respeitado o esquema JSON. Era a única componente verificável — dar 1% por um juízo não verificável seria repor pela porta das traseiras a estimativa que esta extensão veio eliminar.

3. Verificação

Área	Testes	O que garante
Infraestrutura	18	Tabelas, INSERT OR IGNORE em test_id duplicado, incremento de times_passed/times_failed, embedding com e sem sentence-transformers, cosine_similarity em casos degenerados
Geração	7	Testes nos 3 níveis, JSON inválido devolve lista vazia sem lançar, JSON embrulhado é lido, herdados aparecem na lista
Execução	7	pass, fail, falha esperada nos dois sentidos, timeout que não bloqueia o teste seguinte, registo em test_results
Lógica de níveis	14	Nível por iteração, acumulação ligada e desligada, e a fórmula da percentagem em todos os pontos
Herança entre ciclos	4	Ciclo aprovado é herdado, reprovado não, tarefa dissimilar não herda, limite respeitado
HIPPOCAMPUS	3	recommend_tests com ML desligado, com embedding None, e que nunca lança
Schema estendido	3	Defaults quando os campos faltam, leitura quando estão presentes, tipos errados não rebentam
Regressão da base	14	Com a extensão desligada: ciclo corre como antes, nenhuma chamada à geração ou execução, percentagem estimada pelo modelo, breakdown a null

A instrução 61 pedia confirmação de que os 234 testes existentes continuam a passar. Corri a suite em duas partes para o verificar sem margem para dúvida: **234 passam sem o ficheiro novo, 70 passam no ficheiro novo, 304 no total**. Nenhum teste existente mudou de comportamento.

4. O que não fiz — e porquê

Item	Razão
Não activei a extensão	SANDBOX_TESTS_ENABLED=false é o default especificado, e é o correcto: activá-la sem um modelo real a gerar testes produziria zero testes e uma percentagem de 0% em todos os ciclos.
Não removi a execução do código de produção	Com a extensão ligada, a sandbox continua a correr as secções como antes, além dos testes estruturados. Custa pouco e dá aos dois modelos a mesma evidência na validação cruzada.
Não implementei o ranking por qualidade histórica	A especificação descreve-o como caminho futuro para quando o HIPPOCAMPUS tiver dados. Os campos times_used/times_passed/times_failed já são alimentados, portanto os dados vão acumulando; o comentário no código explica-o.
Não marquei testes para revisão	A especificação menciona marcar os que falham sistematicamente. As contagens que o permitem já são registadas, mas a política de revisão não estava definida e inventá-la seria decidir por ti.

5. O que vou fazer a seguir

Nada mudou no que bloqueia: **o sistema continua sem nunca ter corrido com um modelo verdadeiro**. Esta extensão torna o piloto ainda mais decisivo, porque acrescenta uma pergunta nova que só um modelo real responde: **os modelos conseguem gerar testes executáveis?** Um teste que não corre na sandbox é inútil, por muito bem descrito que esteja.

#	Passo	O que mede
1	Piloto com a sandbox desligada	Valida o ciclo base com modelos reais, sem a variável nova a confundir o diagnóstico
2	Ligar a sandbox e medir a taxa de testes executáveis	Quantos dos testes gerados correm de facto — é o número que decide se a extensão é utilizável com os modelos escolhidos
3	Observar a progressão dos níveis ao longo das iterações	Se o código chega ao nível 3, ou se estagna no 1 — e se SANDBOX_MAX_ITER_PER_LEVEL=3 é generoso ou apertado
4	Acumular ciclos aprovados e medir a herança	Quantos testes são herdados e se o limiar de similaridade de 0.75 é o adequado

Continuo a precisar só de três coisas: URL do servidor de inferência, token e id do modelo.

Relatório gerado no fim da etapa. Toda a verificação da secção 3 foi executada; a extensão nunca foi exercitada contra um modelo real.